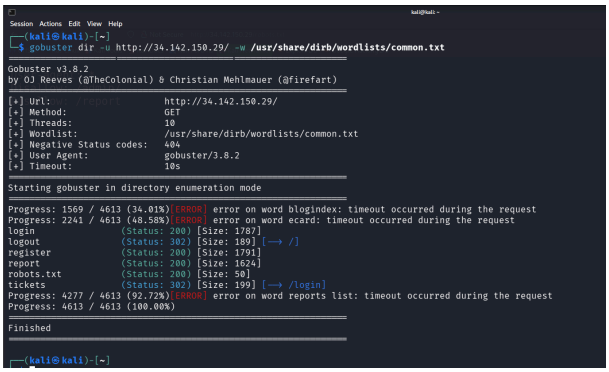
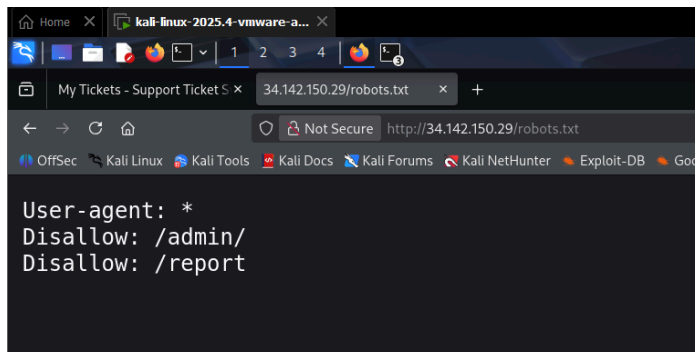
Report 1: Stored Cross-Site Scripting (XSS)

Executive Summary The goal of this assignment was to identify and exploit a Stored XSS vulnerability on the target web application (Problem 2). I discovered that the support ticket system did not properly sanitize user input. By injecting a malicious script into a ticket and forcing the admin bot to view it, I was able to exfiltrate the contents of the admin dashboard and retrieve the hidden flag.

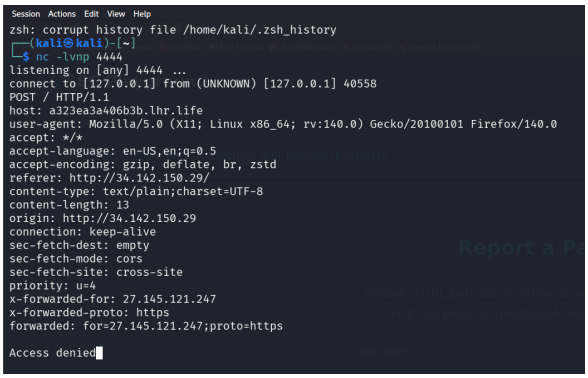
Technical Findings

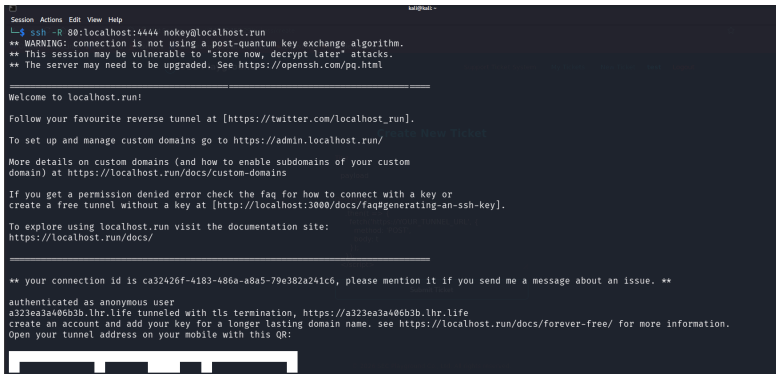
- **Vulnerability Discovery:** I performed a directory scan using `gobuster`, which revealed a `/robots.txt` file. Inside, I found restricted paths for `/admin/` and `/report`.

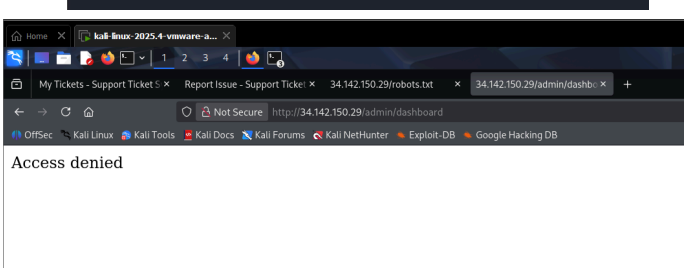


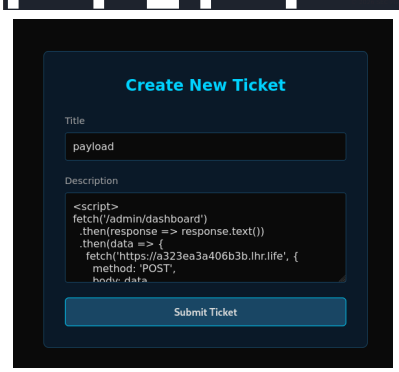


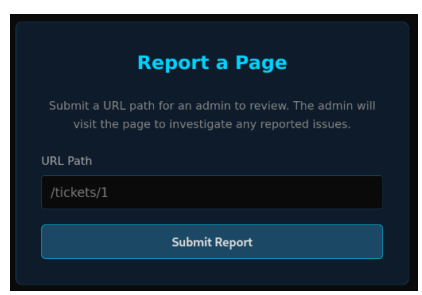
- **Initial Attempt (ncat & localhost.run):** I first tried to capture the data using a local listener. I ran `nc -lvp 4444` and used `localhost.run` to create a public tunnel to my local machine. I submitted a script designed to send data to my tunnel URL. While the tunnel connected, I realized that using a dedicated webhook is more reliable for capturing the full HTML output of the admin dashboard (netcat didn't work for report).



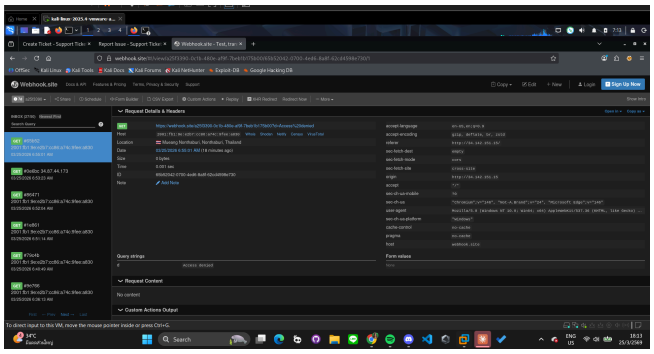


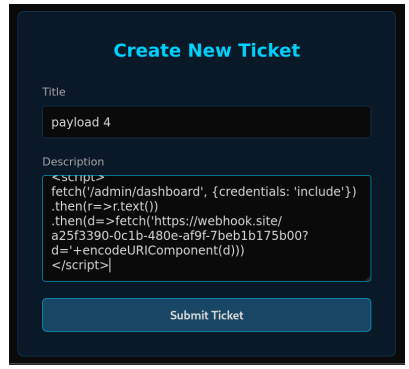


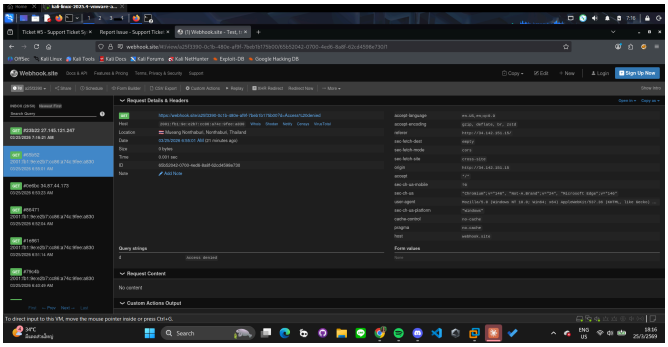


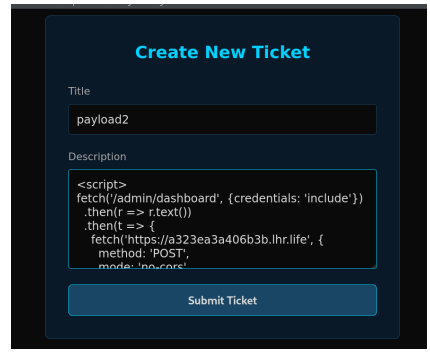


- **Final Exploitation (Webhook):** I switched to using a webhook `.site` URL. I crafted a JavaScript payload using the `fetch` API to get the content of `/admin/dashboard` (including the admin's session credentials) and send that data as a URL-encoded parameter to my webhook.

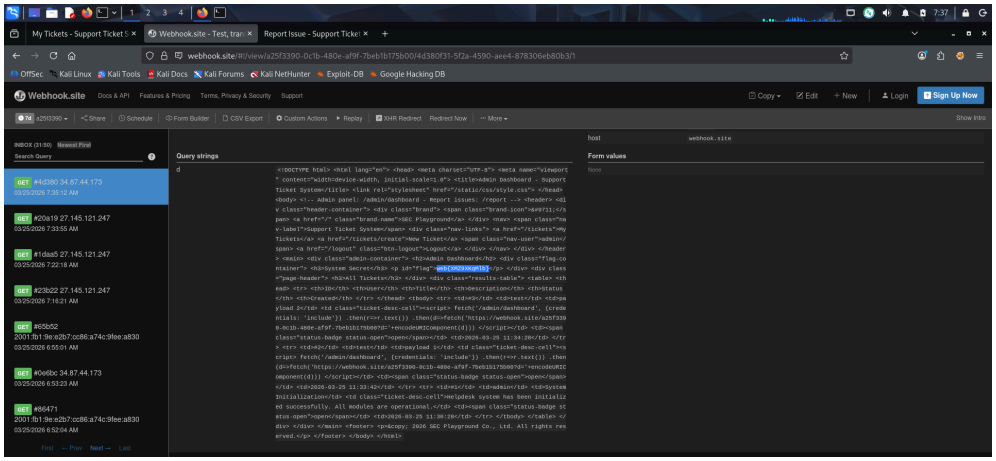








- **Execution:** After submitting the ticket with the payload, I used the `/report` feature to provide the ticket ID to the admin. Once the admin bot visited the page, the script triggered, and the dashboard HTML appeared in my webhook logs.



Impact

- **Severity:** Critical
- **Reasoning:** This vulnerability allows an attacker to execute code in the browser of any user who views the ticket. Since an administrator viewed the ticket, I gained full access to administrative information. This could lead to account takeover or total site compromise.

Remediation

- **Input Sanitization:** The application should filter out characters like < > " ' from the ticket description.
- **Output Encoding:** Use a library to encode data before it is rendered on the page so the browser treats it as plain text instead of executable code.